
CRISP

Creative Rust Implemented Scripting Paradigm

A Modern Scripting Language

Peter Leukanič

Author

July 2026

Version 0.1.5

"Keep your code CRISP and clean."

Contents

Contents	1
1 Introduction	3
2 A First Program	4
2.1 A Temperature Converter	4
2.2 A Temperature Table	5
2.3 An Interactive Temperature Converter	6
3 Getting Started	9
3.1 Installation and Running	9
3.2 Running CRISP Scripts	9
3.3 REPL Features	9
4 Variables and Types	10
4.1 Value Types	10
4.2 Variable Declaration and Assignment	10
4.3 Sigils	10
4.4 Type Introspection and Conversion	11
5 References and Pointer Semantics	12
5.1 Creating and Dereferencing	12
5.2 The Reference Sigil (&)	12
5.3 References to Collections	13
5.4 References as Function Arguments	13
5.5 Reference Semantics: Shallow vs. Deep	14
5.6 Chaining References	14
5.7 Use Cases	15
5.8 Internal Mechanism	15
5.9 Limitations and Caveats	16
5.10 Comparison with Other Languages	16
6 Binary Expressions and Operators	17
6.1 Operator Precedence Table	17
6.2 Arithmetic Operators	17
6.3 String Concatenation	18
6.4 Comparison Operators	18
6.5 Spaceship Operator	19
6.6 Logical Operators	19
6.7 Range Operator	20
6.8 Regex Operators	21
6.9 Assignment Operator	22
6.10 Ternary Operator	22
7 Control Flow	24
7.1 Conditional Statements	24
7.2 While Loops	24
7.3 For Loops	25
7.4 Break and Continue	25
7.5 Control Flow and Try/Finally	26
7.6 Return Statement	27
8 Functions and Closures	29
8.1 Named Functions	29
8.2 Function Parameters	29

8.3	Recursion	30
8.4	Lambda Functions	31
8.5	Closures	31
8.6	Functions as Values	32
9	String Processing	33
9.1	String Reversal	33
9.2	Palindrome Checker	33
9.3	Word Counter	34
9.4	Character Frequency Analyzer	35
10	Error Handling	37
10.1	Try/Catch/Finally	37
10.2	Throw and Die	38
10.3	Control Flow Through Try/Catch	39
10.4	Assert	40
10.5	Exit	41
11	Pattern Matching	43
11.1	Basic Matching	43
11.2	Pattern Types	43
11.3	Where Guards	44
11.4	Multi-Statement Arm Bodies	45
11.5	Break and Continue in Match Arms	46
12	Regular Expressions	47
12.1	Match Operator	47
12.2	Global Match	47
12.3	Substitution	47
13	Object-Oriented Programming	49
13.1	Defining a Class	49
13.2	Creating Objects	49
13.3	Class.method() Auto-Instantiation	49
13.4	Inheritance	50
13.5	Super Calls	50
13.6	Static Methods	51
14	Modules	53
15	System Programming with POSIX	54
15.1	Loading the POSIX Module	54
15.2	Process Management	54
15.3	File Operations (POSIX)	55
15.4	Signals	55
15.5	Users and Environment	56
15.6	POSIX Function Reference	57
16	Console Input and Output	58
16.1	Output Functions	58
16.2	Input Functions	58
16.3	Flush Semantics	59
16.4	Type Conversion Functions	60
17	Multi-Dimensional Arrays	61
17.1	Nested Array Literals	61
17.2	Chained Indexing	61
17.3	Nested Iteration	62

18 JSON Serialization	63
18.1 Serializing to JSON	63
18.2 Deserializing from JSON	64
18.3 Validation	64
18.4 A Complete Example: Configuration Loader	65
18.5 JSON as a Data Exchange Format	66
19 Time and Date Functions	67
19.1 Getting the Current Time	67
19.2 Formatting Dates	68
19.3 Parsing Dates	68
19.4 A Complete Example: Elapsed Time Calculator	69
19.5 Date Arithmetic	70
20 Network Programming	71
20.1 Fetching Web Resources	71
20.2 Sending Data	71
20.3 URL Encoding and Decoding	72
20.4 A Complete Example: Simple Web Client	72
20.5 Network Function Reference	73
21 Performance	74
22 Comparison with Other Languages	75
23 Future Directions	76
24 Conclusion	77
25 Standard Library Reference	78
25.1 Console I/O	78
25.2 Error Handling	78
25.3 Process Control	78
25.4 Type Conversion and Introspection	78
25.5 Array Operations	78
25.6 Math and Random	79
25.7 File I/O (High-Level)	79
25.8 JSON	79
25.9 Crypto	79
25.10Time	79
25.11Network	80
25.12Regex (stdlib)	80
25.13Testing	80
25.14Method Dispatch (Value Methods)	80
25.15POSIX Module (Opt-in via <code>use posix;</code>)	80
25.16Process Management (stdlib)	80
26 Keyword Reference	81
26.1 Control Flow Keywords	83

Abstract

CRISP (Creative Rust Implemented Scripting Paradigm) is a modern scripting language that combines the expressiveness of Perl 5 with the safety and performance of Rust. This paper presents the formal design, implementation, and evaluation of CRISP. Unlike traditional scripting languages, CRISP offers native regular expressions, lambda functions with return support, automatic hash key conversion, field access via dot notation, and full POSIX system call integrationall while maintaining memory safety guarantees inherited from its Rust implementation. We discuss the language architecture, standard library, and performance characteristics, and provide empirical evidence of its viability as a replacement for Perl, Python, and Bash in system scripting contexts.

Core Principles

- (1) **Perl Expressiveness:** Regular expressions, built-in functional tools, and concise syntax for text processing.
- (2) **Rust Safety:** Memory safety, zero-cost abstractions, and fearless concurrency inherited from Rust.
- (3) **Scripting Agility:** One-liners, REPL, quick prototyping, and interactive development.
- (4) **System Access:** Full POSIX system calls, file operations, and process management.

1 Introduction

The landscape of scripting languages is dominated by a trade-off between expressiveness and safety. Languages like Perl offer powerful text processing and concise syntax but lack memory safety guarantees. Languages like Python provide safety through garbage collection but incur performance overhead. Rust offers safety and performance but has a steep learning curve and is not designed for scripting.

CRISP (Creative Rust Implemented Scripting Paradigm) addresses this gap. It provides:

- **Perl-inspired syntax:** Regular expressions (`= s///`), sigils (`$`, `@`, `%`), and built-in functional tools (`map`, `filter`, `grep`)
- **Rust-powered safety:** Memory safety, no segmentation faults, use-after-free protection
- **Scripting agility:** REPL, one-liners, quick prototyping, file execution
- **System access:** Full POSIX system calls via opt-in `use posix;` module
- **Modern error handling:** `try/catch/finally` with `throw/die` keywords
- **Pattern matching:** Rust-style `match` with `where` guards
- **Object-oriented programming:** Classes, constructors, inheritance, static methods, and `super` calls

Definition: CRISP

CRISP is a tree-walking interpreted scripting language implemented in Rust, characterized by:

- Perl-inspired syntax with optional sigils (`$scalar`, `@array`, `%hash`, `&ref`)
- Native regular expressions with `match` (`=`) and substitution (`s///`)
- Lambda functions with explicit `return` and expression bodies
- Automatic hash key conversion and dot-notation field access
- Full POSIX system call integration (opt-in, namespaced)
- Console I/O with type-safe input functions and proper flush semantics
- Loop control flow with `break` and `continue`
- Variable assignment without redeclaration (`x = x + 1`)
- Classes with constructors, inheritance, and static methods

2 A First Program

The only way to learn a new programming language is by writing programs in it. The first program to write is the same for all languages:

Hello, World!

```
say "Hello, World!";
```

Save this in a file called `hello.csp` and run it:

```
$ crisp hello.csp
Hello, World!
```

This program does two things: it calls the built-in function `say`, which prints its argument followed by a newline, and it passes a string literal `"Hello, World!"` as that argument. The semicolon terminates the statement, though CRISP is forgiving; semicolons are optional in many positions.

Thinking in CRISP: The First Step

Every CRISP program is a sequence of statements. A statement can be a variable declaration, a function call, a control-flow construct, or an expression. The simplest useful program is one that produces output and that's exactly what `say` does. As you progress through this book, you'll build on this foundation: from output, to variables, to computation, to control flow, and finally to complex programs that manipulate data, talk to the operating system, and solve real problems.

— *With apologies to Brian Kernighan and Dennis Ritchie, whose "hello, world" started it all.*

2.1 A Temperature Converter

Let's write something more useful. We'll convert Fahrenheit to Celsius, a classic example from the C programming tradition.

The formula is:

$$C = \frac{5}{9} \times (F - 32)$$

Here is a complete CRISP program that reads a Fahrenheit temperature and prints the Celsius equivalent:

Fahrenheit to Celsius Converter (fahrenheit_to_celsius.csp)

```
# fahrenheit_to_celsius.csp  Convert Fahrenheit to Celsius

say "Fahrenheit to Celsius Converter";
say "===== ";
say " ";

say "Enter temperature in Fahrenheit:";
let f = float(readline());

if f == null {
    say "That's not a valid number!";
    exit(1);
}

let c = (5.0 / 9.0) * (f - 32.0);

say f + "°F = " + c + "°C";
```

Let's walk through this program line by line:

- Lines starting with # are comments—they're ignored by CRISP but help humans understand the code.
- `say` prints to the console with a newline; empty `say ""` produces a blank line.
- `readline()` reads a line of text from the user. The program waits for the user to type something and press Enter.
- `float(...)` converts the string to a floating-point number. If the conversion fails (the user typed "hello" instead of "72"), it returns `null`.
- `if f == null` checks for invalid input and exits with an error code if something went wrong.
- The formula $(5.0 / 9.0) * (f - 32.0)$ computes Celsius. Note the use of 5.0 and 9.0 using integers 5 / 9 would produce 0 (integer division), but floats give the correct 0.555...
- Finally, the result is printed with both the original and converted values.

Programmer's Note: Integer vs. Float Division

CRISP's division operator `/` always produces a **Float** result, even when both operands are integers. So `5 / 9` returns `0.555...` (Float), not `0` (truncated integer). This differs from C and Python 2, where integer division truncates. It matches the behavior of Python 3 and modern Perl.

If you want integer division, use the modulo operator `%` and subtract: $(a - a \% b) / b$.

2.2 A Temperature Table

Our first program converts one temperature. A more useful version would print a table. Let's extend it to compute Fahrenheit, Celsius, and Kelvin for a range of values:

Temperature Table (temperature_table.csp)

```
# temperature_table.csp  Print a Fahrenheit-Celsius-Kelvin table

say "Fahrenheit    Celsius    Kelvin";
say "-----    -"

let lower = 0;
let upper = 300;
let step = 20;

let f = lower;
while f <= upper {
  let c = (5.0 / 9.0) * (f - 32.0);
  let k = c + 273.15;

  # Format each row with aligned columns
  say f + "          " + c + "          " + k;

  f = f + step;
}
```

This program introduces several new concepts:

- **Variables:** `lower`, `upper`, `step`, `f`, `c`, `k` store values that change during execution.
- **The while loop:** `while f <= upper { ... }` repeats the body as long as the condition is true.
- **Reassignment:** `f = f + step` increments `f` by 20 each iteration. This is how loops advance without it, the loop would run forever.

Thinking in CRISP: Loop Design

Every loop has three parts: **initialization** (setting `lower`, `upper`, `step` before the loop), **condition** (`f <= upper`), and **increment** (`f = f + step`). Forgetting the increment is the most common loop bug; the program will run forever because the condition never becomes false. CRISP doesn't stop you; it will happily loop until you kill the process with Ctrl+C.

A safer pattern for numeric iteration is the `for` loop with ranges, which handles all three parts automatically: `for f in 0..301 by 20 { ... }`.

2.3 An Interactive Temperature Converter

Let's combine input, computation, and control flow into a full interactive program that converts between Fahrenheit, Celsius, and Kelvin:

Interactive Temperature Converter (temp_convert.csp)

```
# temp_convert.csp  Interactive F/C/K converter

say "Temperature Converter";
say "===== ";
say "Type 'quit' to exit.";
say "";

while true {
  say "Enter temperature (e.g., 100F, 37C, 300K):";
  let input = readline("> ");

  if input == "quit" || input == "exit" {
    say "Goodbye!";
    break;
  }

  if input == "" { continue; }

  # Extract the scale from the last character
  let scale = input[input.len() - 1];
  let value_str = input[0..input.len() - 1];
  let value = float(value_str);

  if value == null {
    say "Invalid number. Try again.";
    continue;
  }

  if scale == "F" || scale == "f" {
    let c = (5.0 / 9.0) * (value - 32.0);
    let k = c + 273.15;
    say value + "°F = " + c + "°C = " + k + " K";
  } else if scale == "C" || scale == "c" {
    let f = (9.0 / 5.0) * value + 32.0;
    let k = value + 273.15;
    say value + "°C = " + f + "°F = " + k + " K";
  } else if scale == "K" || scale == "k" {
    let c = value - 273.15;
    let f = (9.0 / 5.0) * c + 32.0;
    say value + " K = " + c + "°C = " + f + "°F";
  } else {
    say "Unknown scale '" + scale + "'. Use F, C, or K.";
  }

  say "";
}
```

This program demonstrates CRISP's string slicing (`input[0..input.len() - 1]` extracts everything except the last character), the `break` statement (exits the infinite loop), `continue` (skips to the next iteration), and multi-way branching with `if/else if/else`.

What We Learned

From three temperature programs, we've covered:

- **Output:** `say` prints to the console
- **Input:** `readline()` reads user input; `float()` converts strings to numbers
- **Variables:** `let` declares them, `=` reassigns them
- **Arithmetic:** `+`, `-`, `*`, `/` with natural precedence
- **Conditionals:** `if/else if/else` for decision-making
- **Loops:** `while` for repetition; `break` to exit; `continue` to skip
- **String slicing:** `s[0..n]` extracts substrings
- **Error handling:** checking for `null` after conversion
- **Program structure:** comments, whitespace, and logical flow

These are the fundamentals of programming in any language. Everything else—functions, arrays, hashes, classes, regex—builds on this foundation.

3 Getting Started

3.1 Installation and Running

CRISP is a single binary. Build from source:

```
git clone https://github.com/user/crisp.git
cd crisp
cargo build --release
./target/release/crisp
```

3.2 Running CRISP Scripts

CRISP supports four modes of operation:

Table 1: CRISP Execution Modes

Mode	Command	Description
REPL	<code>crisp</code>	Interactive shell
Script file	<code>crisp script.csp</code>	Execute a <code>.csp</code> file
One-liner	<code>crisp -e "say 42"</code>	Inline code execution
AST dump	<code>crisp -ast script.csp</code>	Print parsed AST (debugging)

Additional CLI flags:

- `-debug` Run with verbose debug output
- `-repl` Force REPL mode
- `-version` Print version information

3.3 REPL Features

The interactive REPL supports:

- **Multiline input** Blocks and expressions span multiple lines
- **Debug toggle** `debug verbose` / `debug off` within the REPL
- **Exit** Type `exit` to quit
- **Help** Type `help` for available commands

REPL Session

```
$ crisp
> CRISP v0.1.5 - 2026
Creative Rust Implemented Scripting Paradigm
Perl-inspired . Rust-powered

Type 'exit' to quit, 'help' for commands

>>> let name = "CRISP";
>>> say "Hello, ", name, "!";
Hello, CRISP!
>>> exit
```

4 Variables and Types

4.1 Value Types

CRISP supports dynamic typing with the following value types:

Table 2: CRISP Value Types

Type	Rust Type	Example	Description
Null	<code>Value::Null</code>	<code>null</code>	Absence of value
Bool	<code>Value::Bool</code>	<code>true, false</code>	Boolean values
Int	<code>Value::Int(i64)</code>	<code>42, -7</code>	64-bit signed integer
Float	<code>Value::Float(f64)</code>	<code>3.14, -0.5</code>	64-bit floating point
Str	<code>Value::Str(Rc<String>)</code>	<code>"hello"</code>	UTF-8 string, reference-counted
Array	<code>Value::Array(Rc<RefCell<Vec>>)</code>	<code>[1, 2, 3]</code>	Dynamic, mutable array
Hash	<code>Value::Hash(Rc<RefCell<HashMap>>)</code>	<code>{name => "John"}</code>	String-keyed dictionary
Object	<code>Value::Object</code>	<code>new Point(3, 4)</code>	OOP object with fields and methods
Class	<code>Value::Class</code>	<code>class Point { ... }</code>	Class definition (stored value)
Ref	<code>Value::Ref(Rc<RefCell<Value>>)</code>	<code>\expr</code>	Shared mutable reference
Function	<code>Value::NativeFn</code> or <code>Value::UserFn</code>	<code>fn(x) { ... }</code>	Callable functions

4.2 Variable Declaration and Assignment

Variables are declared with `let` and can be reassigned with `=`:

Variable Declaration and Assignment

```
# Declaration
let name = "CRISP";
let count = 0;
let @numbers = [1, 2, 3];

# Assignment (reassign existing variable)
count = count + 1;
name = "New Name";

# Assignment to undeclared variable is an error
# x = 5; # Error: Undefined variable: x

# Constants
const PI = 3.14159;
# PI = 3.0; # Would be an error (immutable)
```

4.3 Sigils

CRISP supports Perl-inspired optional sigils for type annotation:

Table 3: CRISP Sigils

Sigil	Name	Example	Description
\$	Scalar	\$name, \$count	Single value (string, number, bool)
@	Array	@items, @numbers	Ordered list
%	Hash	%config, %person	Key-value dictionary
&	Reference	&callback, &fn_ref	Reference to a value

Sigils are optional CRISP infers the type from the value:

Sigil Usage

```
# With sigils (Perl style)
let $name = "CRISP";
let @fruits = ["apple", "banana"];
let %person = { name => "John", age => 30 };
let &callback = |x| => x * 2;

# Without sigils (modern style)
let name = "CRISP";
let fruits = ["apple", "banana"];
let person = { name => "John", age => 30 };
let callback = |x| => x * 2;
```

4.4 Type Introspection and Conversion

Type Functions

```
# Get the type of a value
say type(42);           # "int"
say type("hello");      # "string"
say type([1, 2, 3]);    # "array"
say type({ a => 1 });    # "hash"
say type(null);         # "null"
say type(true);         # "bool"

# Parse string to number
let n = int("42");       # Int(42)
let f = float("3.14");   # Float(3.14)

# Invalid parsing returns null
let bad = int("hello");  # null
if bad == null {
  say "Not a number!";
}

# Length of collections
let len = len("CRISP");  # 5
let len2 = len([1, 2, 3]); # 3
let len3 = len({ a => 1, b => 2 }); # 2
```

5 References and Pointer Semantics

CRISP provides shared mutable references through a pointer-like abstraction. Unlike C pointers (raw memory addresses) or C++ references (non-nullable aliases), CRISP references use reference-counted interior mutability `safe`, shared ownership of a mutable value.

5.1 Creating and Dereferencing

A reference is created with the `\` operator and dereferenced with `^`:

Creating and Dereferencing

```
# Create a reference to a value
let x = 42;
let r = \x;

# Dereference to read
say ^r;      # 42

# Dereference to write (modify through reference)
^r = 100;
say x;       # 100  the original was mutated!

# Multiple references to the same value
let a = 10;
let ref1 = \a;
let ref2 = \a;

^ref1 = 20;
say ^ref2;   # 20  both references point to the same cell
say a;       # 20
```

5.2 The Reference Sigil (&)

Variables holding references can be declared with the `&` sigil for clarity:

Reference Sigil

```
# With sigil (explicit "this is a reference")
let &counter = \0;
^counter = ^counter + 1;
say ^counter;   # 1

# Without sigil (implicit works the same)
let counter = \0;
^counter = ^counter + 1;
say ^counter;   # 1

# The sigil is a naming convention, not a type constraint
let &r = \0;
r = \100;       # reassign to reference a different value
say ^r;         # 100
```

5.3 References to Collections

References to arrays and hashes enable shared mutable access to data structures without copying:

References to Collections

```
# Reference to an array
let @data = [1, 2, 3];
let &r = \@data;

push(^r, 4);
push(^r, 5);

say @data;      # [1, 2, 3, 4, 5]
say ^r;         # [1, 2, 3, 4, 5]

# Reference to a hash
let %config = { debug => false };
let &cfgref = \%config;

# Modify through reference
let h = ^cfgref;
h["debug"] = true;

say %config["debug"]; # true
```

5.4 References as Function Arguments

Passing a reference to a function enables the function to mutate the caller's data a form of pass-by-reference:

References as Arguments

```
# Function that mutates through a reference
fn increment(&ref) {
    ^ref = ^ref + 1;
}

let x = 0;
increment(\x);
increment(\x);
increment(\x);
say x; # 3

# Swap two values through references
fn swap(&a, &b) {
    let temp = ^a;
    ^a = ^b;
    ^b = temp;
}

let first = "hello";
let second = "world";
swap(\first, \second);
say first; # world
say second; # hello
```

5.5 Reference Semantics: Shallow vs. Deep

A reference points to the *same* value, not a copy. This means changes through any reference are visible through all others:

Reference Semantics

```
# No reference independent copies
let a = [1, 2, 3];
let b = a; # shallow copy of the array handle
push(b, 4);
say a; # [1, 2, 3] unaffected
say b; # [1, 2, 3, 4]

# With reference shared ownership
let c = [1, 2, 3];
let &rc = \c;
let &rc2 = \c; # second reference to same array
push(^rc, 4);
say ^rc2; # [1, 2, 3, 4] visible through rc2
say c; # [1, 2, 3, 4] original is mutated
```

5.6 Chaining References

References can be nested a reference to a reference:

Chained References

```
let x = 42;
let r1 = \x;      # r1  x
let r2 = \r1;     # r2  r1  x

# Double dereference
say ^^r2;         # 42

# Modify through chain
^^r2 = 99;
say x;            # 99
```

5.7 Use Cases

Table 4: Reference Use Cases

Use Case	Example	Why References
Out-parameters	<code>fn read_config(&out)</code>	Function mutates caller's variable
Shared state	<code>let &state = \@log;</code>	Multiple closures access same data
Avoid copies	<code>let &big = \@large_array;</code>	Pass without cloning large structures
Event handlers	<code>counter.increment(&state)</code>	Multiple handlers mutate shared state
Circular references	<code>let %node = { next => \other }</code>	Build linked structures

5.8 Internal Mechanism

CRISP references are implemented as `Value::Ref(Rc<RefCell<Value>)`:

Table 5: Reference Internals

Component	Rust Type	Role
Rc	<code>std::rc::Rc</code>	Reference counting tracks how many references exist
RefCell	<code>std::cell::RefCell</code>	Interior mutability allows mutation through shared reference
Value	<code>Value</code>	The pointed-to value (can be any CRISP type)

Design Note: Why `Rc<RefCell<Value>`?

The combination of `Rc` and `RefCell` provides two guarantees:

- **Shared ownership (`Rc`):** Multiple references can coexist. The value is deallocated only when the last reference is dropped. No dangling pointers.
- **Interior mutability (`RefCell`):** Mutation through a shared reference is safe because `RefCell` enforces Rust’s borrow rules at runtime (not compile time). Attempting to mutate while another reference is borrowed causes a runtime panic rather than undefined behavior.

This is the same pattern used by Rust’s own `Rc<RefCell<T>` for shared mutable state in single-threaded contexts. CRISP is currently single-threaded, so `Rc` (not `Arc`) is sufficient.

5.9 Limitations and Caveats

- **No automatic dereferencing:** Unlike C++ references or Rust’s `Deref`, CRISP requires explicit `^` to dereference. Writing `r = 5` when `r` is a reference reassigns the reference, not the pointed-to value.
- **Single-threaded only:** `Rc` is not `Send` or `Sync`. CRISP does not currently support multi-threading, so this is not a limitation in practice. Future multi-threaded versions would use `Arc`.
- **No weak references:** All references are strong (`Rc`, not `Weak`). This means circular references will leak memory — they prevent deallocation.
- **Runtime borrow checking:** `RefCell` panics at runtime if you violate borrow rules (e.g., mutating while iterating). This is safer than C’s undefined behavior but less safe than Rust’s compile-time borrow checking.

5.10 Comparison with Other Languages

Table 6: Reference/Pointer Models Across Languages

Language	Create	Access	Safety	Notes
CRISP	<code>\x</code>	<code>^r</code>	Runtime (<code>RefCell</code>)	Shared ownership, interior mutability
C	<code>&x</code>	<code>*p</code>	None	Raw pointer, UB on misuse
C++	<code>&x</code>	Implicit	None	Non-nullable alias
Rust	<code>&x</code> / <code>&mut x</code>	Implicit	Compile-time	Borrow checker
Perl	<code>\\$x</code>	<code>\$\$r</code> / <code>\$r-></code>	Runtime (refcount)	Garbage collected
Python	Implicit	Implicit	Runtime (GC)	Everything is a reference

CRISP’s model sits between Perl (explicit `\/$` deref, reference-counted) and Rust (safe interior mutability pattern), with the explicitness of C pointers but the safety of runtime borrow checking.

6 Binary Expressions and Operators

CRISP provides a rich set of binary operators organized by precedence. The parser uses a Pratt parsing algorithm with defined binding powers for each operator, eliminating ambiguity in complex expressions.

6.1 Operator Precedence Table

Operators are evaluated from highest to lowest binding power. Higher in the table = tighter binding:

Table 7: Operator Precedence Highest to Lowest

Precedence	Operators	Associativity	Description
1 (highest)	**	Right	Exponentiation
2	* / %	Left	Multiplication, division, modulo
3	+ -	Left	Addition, subtraction, string concatenation
4	..	Non-associative	Range construction
5	< > <= >= <=>	Non-associative	Comparison, spaceship
6	== != = !	Non-associative	Equality, regex match
7	&&	Left	Logical AND
8		Left	Logical OR
9 (lowest)	=	Right	Assignment

6.2 Arithmetic Operators

Arithmetic Operators

```
# Basic arithmetic
let sum = 10 + 5;      # 15
let diff = 10 - 5;     # 5
let prod = 10 * 5;     # 50
let quot = 10 / 5;     # 2
let mod = 10 % 3;      # 1

# Exponentiation (right-associative)
let pow = 2 ** 3;      # 8
let chain = 2 ** 3 ** 2; # 512 (2 ** (3 ** 2) = 2 ** 9)

# Mixed types: Int + Float  Float
let mixed = 10 + 3.5;   # 13.5 (Float)

# Division always produces Float
let half = 10 / 4;      # 2.5 (Float, not truncated)

# Negative numbers
let neg = -5 + 3;       # -2
```

6.3 String Concatenation

The `+` operator concatenates strings. At least one operand must be a string; the other is converted via `.as_str()`:

String Concatenation

```
# String + String
let greeting = "Hello, " + "World!";
say greeting; # Hello, World!

# String + Number
let msg = "Value: " + 42;
say msg; # Value: 42

# String + Bool
say "Flag: " + true; # Flag: true

# Number + String works too
say 100 + " points"; # 100 points

# Chaining
let full = "CRISP" + " v" + 0 + "." + 1 + "." + 0;
say full; # CRISP v0.1.0
```

6.4 Comparison Operators

Comparison operators return `true` or `false`. They are non-associative chaining them without parentheses is an error:

Comparison Operators

```
# Numeric comparisons
say 10 > 5;           # true
say 10 < 5;           # false
say 10 >= 10;         # true
say 10 <= 5;          # false

# Equality (works across types)
say 10 == 10;         # true
say 10 == 5;          # false
say 10 != 5;          # true
say "hi" == "hi";     # true

# Floats compare as f64
say 3.14 == 3.14;     # true

# Non-associative use && for chaining
# say 1 < x < 10;      # Error!
say 1 < x && x < 10;   # Correct

# Null equality
let x = null;
say x == null;        # true
say null != 42;       # true
```

6.5 Spaceship Operator

The `<=>` (spaceship) operator returns `-1`, `0`, or `1` for less-than, equal, or greater-than:

Spaceship Operator

```
# Numeric comparison
say 3 <=> 5;          # -1 (less)
say 5 <=> 5;          # 0  (equal)
say 7 <=> 5;          # 1  (greater)

# String comparison (lexicographic)
say "apple" <=> "banana"; # -1
say "crisp" <=> "crisp";  # 0
say "zebra" <=> "apple";  # 1

# Useful for sorting callbacks
let sorted = sort(arr, |a, b| => a <=> b);
```

6.6 Logical Operators

`&&` (AND) short-circuits if the left operand is falsy, the right is never evaluated. `||` (OR) short-circuits if the left operand is truthy, the right is never evaluated:

Logical Operators

```
# AND (ampersand) both must be true
if x > 0 && x < 100 {
    say "x is in range";
}

# OR (pipe) at least one must be true
if name == "admin" || name == "root" {
    say "privileged user";
}

# Short-circuit: expensive() is never called
let x = 0;
if x != 0 && expensive() {
    say "won't happen";
}

# Short-circuit: fallback() is never called
let y = 42;
if y > 0 || fallback() {
    say "short-circuited";
}

# Combined complex conditions
if (age >= 18 && age <= 65) || has_permission {
    say "access granted";
}

# Truthiness: false, null, 0, "" are falsy
if 0 || null || false || "" {
    say "never runs";
} else {
    say "all falsy"; # this runs
}
```

6.7 Range Operator

The `..` operator creates a range. Ranges are used primarily in for loops and array slicing:

Range Operator

```
# Numeric range (inclusive start, exclusive end)
for i in 0..5 {
    say i;
}
# Output: 0 1 2 3 4

# Range stored in variable
let r = 1..10;

# Array slicing with range
let @arr = [10, 20, 30, 40, 50];
say @arr[1..3]; # [20, 30]

# String slicing
say "CRISP"[0..3]; # "CRI"
```

6.8 Regex Operators

CRISP provides `=` (match) and `!` (negated match) for regular expression testing:

Regex Operators

```
let text = "The quick brown fox";

# =~ returns boolean in scalar context
if text =~ /fox/ {
    say "Found fox!";
}

# !~ negated match
if text !~ /dragon/ {
    say "No dragons here";
}

# With g flag, =~ returns array of matches
let words = text =~ /\w+/g;
say words; # ["The", "quick", "brown", "fox"]

# Case-insensitive
if text =~ /FOX/i {
    say "case-insensitive match";
}

# Combine with logical operators
if text =~ /quick/ && text =~ /fox/ {
    say "Contains both quick and fox";
}
```


6.9 Assignment Operator

The `=` operator reassigns an existing variable. It is right-associative, enabling chained assignment:

Assignment Operator

```
# Reassignment (variable must already exist)
let x = 10;
x = 20;      # valid  x exists
x = x + 1;   # valid  self-referencing

# Right-associative chained assignment
let a = 0;
let b = 0;
a = b = 42;
say a; # 42
say b; # 42

# Assignment in expression context
let y = (x = 5) + 10;
say x; # 5
say y; # 15

# Error: assigning to undeclared variable
# z = 5; # Error: Undefined variable: z
```

6.10 Ternary Operator

The ternary `? :` operator provides inline conditional evaluation. It is right-associative and has the lowest precedence above assignment:

Ternary Operator

```
# Basic ternary
let label = score > 50 ? "pass" : "fail";

# Ternary with expressions
let max = a > b ? a : b;

# Nested ternary (right-associative)
let grade = score >= 90 ? "A"
              : score >= 80 ? "B"
              : score >= 70 ? "C"
              : "F";

# Ternary in expressions
say "You " + (age >= 18 ? "may" : "may not") + " enter.";
```

Design Note: Pratt Parsing

CRISP uses a Pratt (top-down operator precedence) parser for binary expressions. Each operator is assigned a left and right binding power:

- **Left binding power:** Determines whether this operator "grabs" the expression on its left from the previous operator. If the previous operator's right power is lower than this operator's left power, this operator wins.
- **Right binding power:** Determines whether this operator "allows" the next operator to grab its right operand. Higher right power = tighter binding on the right.
- **Associativity:** Implemented by making left binding power lower than right (left-associative, e.g., $a - b - c = (a - b) - c$) or equal (right-associative, e.g., $a = b = c = a = (b = c)$).

This approach eliminates the need for explicit precedence tables in the grammar. Precedence is encoded directly in the numeric binding powers.

7 Control Flow

7.1 Conditional Statements

CRISP supports `if`, `else if`, and `else` with block bodies:

Conditional Statements

```
# Basic if/else
let x = 42;
if x > 100 {
    say "Large";
} else {
    say "Small";
}

# if/else if/else chain
let score = 85;
if score >= 90 {
    say "A";
} else if score >= 80 {
    say "B";
} else if score >= 70 {
    say "C";
} else {
    say "F";
}

# Single-expression conditions
let name = "CRISP";
if name == "CRISP" {
    say "It's CRISP!";
}
```

7.2 While Loops

The `while` loop evaluates a condition before each iteration:

While Loops

```
# Basic while loop
let i = 0;
while i < 5 {
  say i;
  i = i + 1;
}
# Output: 0 1 2 3 4

# Infinite loop with break
let count = 0;
while true {
  say count;
  count = count + 1;
  if count >= 5 { break; }
}
# Output: 0 1 2 3 4
```

7.3 For Loops

The `for` loop iterates over arrays, hashes, strings, and ranges:

For Loops

```
# For loop over array
let @fruits = ["apple", "banana", "cherry"];
for fruit in @fruits {
  say fruit;
}
# Output: apple banana cherry

# For loop over range
for i in 0..5 {
  say i;
}
# Output: 0 1 2 3 4

# For loop over hash (iterates values)
let %ages = { alice => 25, bob => 30 };
for age in %ages {
  say age;
}
# Output: 25 30

# For loop over string (iterates characters)
for ch in "CRISP" {
  say ch;
}
# Output: C R I S P
```

7.4 Break and Continue

`break` exits the loop immediately. `continue` skips to the next iteration:

Break and Continue

```
# Break    exit loop early
let i = 0;
while true {
  say i;
  i = i + 1;
  if i >= 5 { break; }
}
# Output: 0 1 2 3 4

# Continue skip to next iteration
for i in 0..10 {
  if i % 2 == 0 { continue; } # skip evens
  say i;
}
# Output: 1 3 5 7 9

# Both together
for i in 0..10 {
  if i == 2 { continue; }
  if i == 7 { break; }
  say i;
}
# Output: 0 1 3 4 5 6
```

7.5 Control Flow and Try/Finally

`break` and `continue` propagate correctly through `try/finally` the `finally` block always executes before the signal propagates:

Break with Finally

```
while true {
  try {
    say "inside try";
    break;
  } finally {
    say "finally runs!";
  }
}
# Output:
# inside try
# finally runs!

# continue also respects finally
for i in 0..3 {
  try {
    if i == 1 { continue; }
    say "processing ", i;
  } finally {
    say "cleanup for ", i;
  }
}
# Output:
# processing 0 / cleanup for 0
# cleanup for 1
# processing 2 / cleanup for 2
```

7.6 Return Statement

`return` exits a function with an optional value. It uses a sentinel error internally to unwind the call stack:

Return Statement

```
# Return with value
fn add(a, b) {
  return a + b;
}

# Early return from condition
fn abs(x) {
  if x >= 0 { return x; }
  return -x;
}

# Return without value (returns null)
fn do_nothing() {
  return;
}
```

Design Note: Control Flow Signals

`return`, `break`, and `continue` are implemented as sentinel errors (`RuntimeError::ReturnSignal`, `BreakSignal`, `ContinueSignal`) that unwind through the call stack. This ensures they propagate correctly through nested structures like `if`, `match`, `try/catch`, and `for` loops. Using `break` or `continue` outside a loop produces a runtime error.

8 Functions and Closures

8.1 Named Functions

CRISP supports named function definition with the `fn` keyword. Functions are first-class values stored in the environment, enabling recursion and self-reference:

Named Functions

```
# Basic function definition
fn greet(name) {
  say "Hello, ", name, "!";
}

greet("CRISP"); # Hello, CRISP!

# Function with return value
fn add(a, b) {
  return a + b;
}

let result = add(3, 7);
say result; # 10

# Early return
fn abs(x) {
  if x >= 0 {
    return x;
  }
  return -x;
}

say abs(-42); # 42
say abs(42); # 42
```

8.2 Function Parameters

Functions accept positional parameters. Each parameter is bound to the corresponding argument at call time:

Function Parameters

```
# Multiple parameters
fn distance(x1, y1, x2, y2) {
  let dx = x2 - x1;
  let dy = y2 - y1;
  return sqrt(dx * dx + dy * dy);
}

# Parameters with sigils (type annotation)
fn process(@arr, %opts) {
  say "Array: ", @arr;
  say "Options: ", %opts;
}

process([1, 2, 3], { debug => true });
```

8.3 Recursion

Functions are recursive with a maximum depth of 1000 calls. The function name is available inside its own body via self-reference:

Recursion

```
# Factorial classic recursion
fn factorial(n) {
  if n <= 1 {
    return 1;
  }
  return n * factorial(n - 1);
}

say factorial(5);    # 120
say factorial(10);   # 3628800

# Fibonacci
fn fib(n) {
  if n <= 1 { return n; }
  return fib(n - 1) + fib(n - 2);
}

say fib(10);    # 55

# Mutual recursion
fn is_even(n) {
  if n == 0 { return true; }
  return is_odd(n - 1);
}

fn is_odd(n) {
  if n == 0 { return false; }
  return is_even(n - 1);
}
```

8.4 Lambda Functions

Lambdas are anonymous functions defined with pipe syntax. They come in two forms: expression-body (implicit return) and block-body (explicit return):

Lambda Functions

```
# Expression-body lambda (implicit return)
let double = |x| => x * 2;
say double(10); # 20

# Block-body lambda (explicit return)
let square = |x| => {
  let result = x * x;
  return result;
};
say square(5); # 25

# Multiple parameters
let add = |a, b| => a + b;
say add(3, 7); # 10

# Lambda as argument
let numbers = [1, 2, 3, 4, 5];
let squared = map(numbers, |n| => n * n);
say squared; # [1, 4, 9, 16, 25]

# Lambda capturing outer variables
let factor = 10;
let scale = |x| => x * factor;
say scale(5); # 50
```

8.5 Closures

Functions capture their lexical environment at definition time. Each function call creates a fresh environment that chains to the captured parent:

Closures

```
# Counter factory returns a closure
fn make_counter(start) {
  return || => {
    let current = start;
    start = start + 1;
    return current;
  };
}

let counter1 = make_counter(0);
let counter2 = make_counter(100);

say counter1(); # 0
say counter1(); # 1
say counter1(); # 2
say counter2(); # 100
say counter2(); # 101

# Each closure has its own captured environment
```

8.6 Functions as Values

Functions are first-class values: they can be stored in variables, passed as arguments, and returned from other functions:

First-Class Functions

```
# Assign function to variable
let greet = fn(name) {
  return "Hello, " + name + "!";
};
say greet("CRISP"); # Hello, CRISP!

# Pass function as argument
fn apply_twice(f, x) {
  return f(f(x));
}

let inc = |x| => x + 1;
say apply_twice(inc, 5); # 7 (inc(inc(5)) = inc(6) = 7)

# Return function from function
fn multiplier(factor) {
  return |x| => x * factor;
}

let triple = multiplier(3);
say triple(10); # 30
```

9 String Processing

CRISP inherits Perl's strength in text manipulation. Strings are first-class values with rich built-in methods and a flexible set of operators. This section presents four classic string-processing programs that demonstrate CRISP's capabilities.

9.1 String Reversal

Reversing a string is the “hello world” of string algorithms. Here are three approaches, from manual to idiomatic:

String Reversal (reverse_string.csp)

```
# reverse_string.csp  Three ways to reverse a string

fn reverse_iterative(s) {
  let result = "";
  let i = s.len() - 1;
  while i >= 0 {
    result = result + s[i];
    i = i - 1;
  }
  return result;
}

fn reverse_for(s) {
  let result = "";
  for ch in s {
    result = ch + result; # prepend each character
  }
  return result;
}

fn reverse_split(s) {
  # split into characters, reverse array, join back
  return s.split("").reverse().join("");
}

say reverse_iterative("CRISP");    # PSIRC
say reverse_for("hello");          # olleh
say reverse_split("world");        # dlrow
```

The iterative version walks backward through the string index by index. The `for`-loop version takes advantage of CRISP's character iteration: each character is prepended to the result, naturally reversing the order. The split-reverse-join pipeline is the most idiomatic; it composes three operations into a single expression.

9.2 Palindrome Checker

A palindrome reads the same forward and backward. This program normalizes input (strips spaces, ignores case and punctuation) before checking:

Palindrome Checker (palindrome.csp)

```
# palindrome.csp  Check if a string is a palindrome

fn is_palindrome(s) {
  # Normalize: lowercase, strip non-alphanumeric characters
  let cleaned = "";
  for ch in s {
    let lower = ch.to_lower();
    if lower =~ /[a-z0-9]/ {
      cleaned = cleaned + lower;
    }
  }

  # Compare with reversed version
  let len = cleaned.len();
  let i = 0;
  while i < len / 2 {
    if cleaned[i] != cleaned[len - 1 - i] {
      return false;
    }
    i = i + 1;
  }
  return true;
}

# Test cases
let tests = [
  "racecar",
  "A man a plan a canal Panama",
  "hello",
  "Madam, I'm Adam",
  "12321",
  "not a palindrome"
];

for test in tests {
  let result = is_palindrome(test) ? "YES" : "NO";
  say test + " " + result;
}
```

Thinking in CRISP: String Normalization

The palindrome checker illustrates a common text-processing pattern: **normalize, then analyze**. Real-world strings are messy—they have spaces, punctuation, mixed case. Rather than making the comparison logic handle all these variations, we first transform the input into a canonical form (lowercase alphanumeric), then apply the simple algorithm. This pattern—sanitize input, then operate—appears everywhere in text processing: search engines strip stop words before indexing, compilers tokenize before parsing, spam filters normalize before classifying. Get comfortable with it; you'll use it often.

9.3 Word Counter

This program counts words, lines, and characters—the classic Unix `wc` utility, in CRISP:

Word Counter (word_count.csp)

```
# word_count.csp  Count words, lines, and characters

fn wc(text) {
    let chars = text.len();

    # Count lines by counting newlines
    let lines = 1; # at least one line
    for ch in text {
        if ch == "\n" {
            lines = lines + 1;
        }
    }

    # Count words: split on whitespace, filter empties
    let raw_words = text.split(" ");
    let words = 0;
    for w in raw_words {
        if w != "" { words = words + 1; }
    }

    return { lines => lines, words => words, chars => chars };
}

# Test with a sample paragraph
let sample = "CRISP combines Perl expressiveness\n" +
             "with Rust safety.\n" +
             "It's a modern scripting language.\n";

let stats = wc(sample);
say "Lines: " + stats.lines;
say "Words: " + stats.words;
say "Chars: " + stats.chars;
```

9.4 Character Frequency Analyzer

This program counts how often each character appears in a string useful for cryptanalysis, text statistics, and data exploration:

Character Frequency (char_freq.csp)

```
# char_freq.csp  Character frequency analysis

fn char_frequency(text) {
  let freq = {};

  for ch in text {
    if ch == " " || ch == "\n" || ch == "\t" {
      continue; # skip whitespace
    }

    # Initialize count if first time seeing this char
    if !freq.contains(ch) {
      freq[ch] = 0;
    }
    freq[ch] = freq[ch] + 1;
  }

  return freq;
}

fn print_freq_table(freq) {
  say "Char | Count | Bar";
  say "-----+-----+-----";

  for ch in freq.keys() {
    let count = freq[ch];
    let bar = "" x count; # repeat block char
    say " " + ch + " | " + count + " | " + bar;
  }
}

let text = "The quick brown fox jumps over the lazy dog";
let freq = char_frequency(text);
print_freq_table(freq);
```

The frequency analyzer builds a hash (`freq`) mapping each character to its count. The `x` operator (string repetition) creates a simple horizontal bar chart. This is a pattern you'll see throughout CRISP: use a hash for counting, an array for ordering, and lambdas for transformation.

Why String Processing Matters

Text is universal. Log files, configuration files, CSV data, HTML pages, JSON APIs, source code—everything is text. The languages that excel at text processing (Perl, AWK, sed) became the backbone of system administration and data wrangling for decades. CRISP inherits this tradition: strings are not an afterthought but a central design element. The four programs in this section—reversal, palindrome checking, word counting, and frequency analysis—are not just exercises. They're the building blocks of real-world text processing: validation, normalization, counting, and analysis. Master them, and you'll find that most text problems reduce to combinations of these patterns.

10 Error Handling

CRISP provides a modern error handling system with `try/catch/finally` blocks, catchable `throw` and `die` keywords, and proper propagation of control-flow signals through error boundaries.

10.1 Try/Catch/Finally

The `try` block executes code that may produce errors. If an error occurs, the optional `catch` block receives it. The `finally` block always executes regardless of errors:

Basic Try/Catch

```
# Basic try/catch
try {
  let x = 1 / 0;
} catch e {
  say "Error: ", e;
}
# Output: Error: Division by zero

# Catch with alias syntax (catch e as err)
try {
  die("something went wrong");
} catch e as err {
  say "Caught: ", err;
}
# Output: Caught: something went wrong
```


Try/Catch/Finally

```
# Finally always executes
try {
  say "trying...";
  die("oops");
} catch e {
  say "error: ", e;
} finally {
  say "cleanup always runs";
}

# Output:
# trying...
# error: oops
# cleanup always runs

# Finally runs even without catch
try {
  say "work";
} finally {
  say "done";
}

# Output: work / done

# Nested error handling
try {
  try {
    throw "inner error";
  } catch e {
    say "inner: ", e;
  }
} catch e {
  say "outer: ", e;
}

# Output: inner: inner error
```

10.2 Throw and Die

Both `throw` and `die` produce catchable `UserError` values. They are interchangeable:

Throw and Die

```
# throw   catchable user error
try {
  throw "invalid configuration";
} catch e {
  say "Problem: ", e;
}

# die    identical behavior, also catchable
try {
  die "fatal condition";
} catch e {
  say "Recovered from: ", e;
}

# Uncaught throw/die terminates the script
throw "unrecoverable";
# Error: unrecoverable
```

10.3 Control Flow Through Try/Catch

`return`, `break`, and `continue` propagate through `try/catch/finally` correctly the `finally` block always executes before the signal continues:

Control Flow with Finally

```
# break respects finally
while true {
  try {
    say "inside";
    break;
  } finally {
    say "cleanup!";
  }
}
# Output: inside / cleanup!

# continue respects finally
for i in 0..3 {
  try {
    if i == 1 { continue; }
    say "processing ", i;
  } finally {
    say "iter ", i;
  }
}
# Output: processing 0 / iter 0 / iter 1 / processing 2 / iter 2

# return from function through finally
fn demo() {
  try {
    return "early";
  } finally {
    say "still runs!";
  }
}
say demo();
# Output: still runs! / early
```

10.4 Assert

The `assert` function verifies a condition and produces an error if it fails:

Assert

```
# Basic assertion
assert(2 + 2 == 4); # passes silently

# Assertion with custom message
let x = -1;
assert(x > 0, "x must be positive");
# Error: x must be positive
```

Design Note: Error Signals

CRISP implements four internal sentinel errors (`RuntimeError::ReturnSignal`, `BreakSignal`, `ContinueSignal`, `ExitSignal`) that unwind through the interpreter's call stack. These are not caught by `catch` blocks only real errors are. The `try/catch/finally` handler distinguishes between them:

- **Real errors:** Captured by `catch`, then `finally` executes
- **Control-flow signals:** Bypass `catch`, `finally` still executes, then the signal is re-emitted
- **ExitSignal:** Propagates through all boundaries including `try/catch` and loops. `finally` blocks execute, then the process terminates with the given exit code. Not catchable by `catch`.

This ensures `break` from inside a `try` block inside a loop works as expected the loop receives the `BreakSignal` after `finally` runs.

10.5 Exit

The `exit()` function terminates the process with an optional exit code. It produces a special `ExitSignal` that is **not catchable** by `try/catch` blocks. However, `finally` blocks still execute before the process terminates:

Exit

```
# Exit with default code 0
exit();

# Exit with specific code
exit(1);

# finally runs before exit
try {
  exit(2);
} finally {
  say "cleanup runs!";
}

# Output: cleanup runs!
# Process exits with code 2

# _exit() skips finally immediate hard exit
_exit(0); # no cleanup, process dies immediately
```

Design Note: `ExitSignal` vs. `_exit`

CRISP provides two exit mechanisms:

- `exit(code?)` Emits `ExitSignal`, which propagates through `finally` blocks before the process terminates. This ensures cleanup code runs. Not catchable by `catch` it always terminates.
- `_exit(code?)` Calls Rust's `std::process::exit` directly. No cleanup, no `finally`, immediate termination. Use when you need a hard stop.

The `ExitSignal` uses the same sentinel-error mechanism as `return/break/continue` to unwind the call stack, ensuring `finally` blocks execute before the interpreter propagates the signal to the top level and calls `std::process::exit`.

11 Pattern Matching

CRISP supports Rust-style pattern matching with the `match` expression. Patterns can be literal values, variable bindings, or wildcards, with optional `where` guards.

11.1 Basic Matching

Basic Match

```
# Simple value matching
let value = 42;
match value {
  0  => { say "zero"; }
  1  => { say "one"; }
  42 => { say "the answer"; }
  _  => { say "something else"; }
}

# Output: the answer

# Default case with wildcard
let x = 99;
match x {
  1  => { say "one"; }
  2  => { say "two"; }
  _  => { say "unknown: ", x; }
}

# Output: unknown: 99
```

11.2 Pattern Types

CRISP supports the following match patterns:

Table 8: Match Pattern Types

Pattern	Syntax	Example	Matches
Integer	42	42 => ...	Exact integer equality
String	"hello"	"hello" => ...	Exact string equality
Boolean	true, false	true => ...	Exact boolean match
Variable	name	n => ...	Any value, binds to name
Wildcard	_	_ => ...	Any value, no binding

Pattern Types

```
# Integer patterns
match 42 {
  0  => { say "zero"; }
  42 => { say "the answer"; }
  _  => { say "other"; }
}

# String patterns
match "hello" {
  "hello" => { say "greeting"; }
  "bye"   => { say "farewell"; }
  _       => { say "unknown"; }
}

# Boolean patterns
match true {
  true  => { say "yes"; }
  false => { say "no"; }
}

# Variable binding matches anything, binds to name
match readline("Enter: ") {
  n => { say "you entered: ", n; }
}
```

11.3 Where Guards

where guards add conditions to pattern arms. The guard expression is evaluated after the pattern matches, and the arm only fires if the guard is true:

Where Guards

```
# Type-based dispatch
let guess = read("Your guess: ");

match guess {
  n where type(n) == "int" => {
    say "Integer: ", n;
  }
  s where s == "" => {
    say "Empty input.";
  }
  - => {
    say "Something else: ", guess;
  }
}

# Range-based guards
let score = 85;
match score {
  s where s >= 90 => { say "A"; }
  s where s >= 80 => { say "B"; }
  s where s >= 70 => { say "C"; }
  s where s >= 60 => { say "D"; }
  -                => { say "F"; }
}

# Output: B

# Complex guard expression
let x = 15;
match x {
  n where n > 0 && n % 2 == 0 => { say "positive even"; }
  n where n > 0                => { say "positive odd"; }
  n where n < 0                => { say "negative"; }
  -                            => { say "zero"; }
}

# Output: positive odd
```

11.4 Multi-Statement Arm Bodies

Each arm body can contain multiple statements. The match expression returns the value of the last statement in the matched arm:

Multi-Statement Arms

```
match value {
  1 => {
    say "processing one...";
    say "done!";
  }
  2 => {
    let doubled = 2 * 2;
    say "doubled: ", doubled;
  }
  _ => {
    say "fallback";
  }
}
```

11.5 Break and Continue in Match Arms

`break` and `continue` propagate correctly from inside `match` arms when the `match` is inside a loop:

Match Inside Loops

```
# break from match inside while
let i = 0;
while true {
  i = i + 1;
  match i {
    n where n >= 5 => { break; }
    _ => { say i; }
  }
}
# Output: 1 2 3 4

# continue from match inside for
for i in 0..5 {
  match i {
    n where n % 2 == 0 => { continue; }
    _ => { say i; }
  }
}
# Output: 1 3
```

Design Note: Match Evaluation

Match arms are evaluated in order from top to bottom. The first arm whose pattern matches and whose guard (if present) evaluates to `true` wins. Only that arm's body executes. There is no fall-through between arms.

For variable-binding patterns with guards, the variable is bound in a temporary scope that includes the guard expression and the arm body. This ensures guard expressions can reference the bound variable without leaking into the outer scope.

12 Regular Expressions

CRISP includes native Perl-style regular expression support via the Rust `regex` crate. The `=` operator tests for a match, and the `s///` syntax performs substitution.

12.1 Match Operator

The `=` match operator returns a boolean in scalar context:

Regex Match

```
let text = "The quick brown fox jumps over the lazy dog";

# Basic match
let result = text =~ /fox/;
say result; # true

# Match in if condition
if text =~ /quick/ {
    say "Found quick!";
}

# Case-insensitive flag
if text =~ /FOX/i {
    say "Found fox!";
}

# Match against a variable pattern
let pattern = "brown";
if text =~ /pattern/ {
    say "Matched!";
}
```

12.2 Global Match

With the `g` flag, `=` returns an array of all matches:

Global Match

```
let text = "cat dog cat bird cat";

# Find all matches
let matches = text =~ /cat/g;
say matches; # ["cat", "cat", "cat"]

# Count occurrences
say len(matches); # 3
```

12.3 Substitution

The `s///` operator replaces matched text. Use `g` for global replacement:

Substitution

```
let text = "The quick brown fox";

# Single substitution (first occurrence)
let result = s/fox/cat/;
say result; # The quick brown cat

# Global substitution
let text2 = "one two one two";
let result2 = s/one/1/g;
say result2; # 1 two 1 two

# Using the Substitute expression directly
let modified = substitute(text, "brown", "red", false);
say modified; # The quick red fox
```

Design Note: Regex Implementation

Regular expressions in CRISP are powered by the Rust **regex** crate, which provides:

- **Compile-time optimization:** Patterns are compiled once and cached
- **Unicode support:** Full UTF-8 matching by default
- **Perl-compatible syntax:** Most common Perl regex features are supported
- **Safety:** No catastrophic backtracking the regex engine uses finite automata

The regex is not a first-class type patterns are passed as strings and compiled at the point of use. Capture groups (\$1, \$2) are not currently implemented as automatic variables but are available through the `regex_capture()` function in the standard library.

13 Object-Oriented Programming

CRISP v0.1.5 introduces Python/Perl-style object-oriented programming with classes, constructors, inheritance, and static methods. OOP in CRISP is prototype-free and class-based, with clean syntax modeled on modern dynamic languages.

13.1 Defining a Class

A class is defined with the `class` keyword. Methods are defined with `fn` inside the class body. The special `new` method is the constructor:

Class Definition

```
class Point {
  fn new(x, y) {
    self.x = x;
    self.y = y;
  }

  fn distance() {
    return sqrt(pow(self.x, 2) + pow(self.y, 2));
  }

  fn describe() {
    return "Point(" + self.x + ", " + self.y + ")";
  }
}
```

13.2 Creating Objects

Objects are created with the `new` keyword, which calls the constructor:

Creating Objects

```
let p = new Point(3, 4);

say p.x;           # 3
say p.y;           # 4
say p.distance();  # 5
say p.describe();  # Point(3, 4)

# Mutate fields directly
p.x = 10;
say p.distance();  # sqrt(100 + 16) = 10.77...
```

13.3 Class.method() Auto-Instantiation

Calling a method on a class (not an instance) auto-creates an object. This is syntactic sugar that calls `new` with the arguments, then calls the method:

Auto-Instantiation

```
# These are equivalent:
let p1 = Point.new(5, 12); # explicit new
let p2 = new Point(5, 12); # contextual new keyword

say p1.distance(); # 13
say p2.distance(); # 13

# Auto-instantiation works for ANY method, not just new:
let p3 = Point.move_to(6, 8); # creates Point(0,0), moves to
                              (6,8)
say p3.describe(); # Point(6, 8)
```

13.4 Inheritance

Classes can extend other classes with the `extends` keyword. Child classes inherit all methods and fields from the parent:

Inheritance

```
class Point3D extends Point {
  fn new(x, y, z) {
    super(x, y); # call parent constructor
    self.z = z;
  }

  fn distance() {
    let d2d = super.distance(); # call parent method
    return sqrt(pow(d2d, 2) + pow(self.z, 2));
  }

  fn describe() {
    return "Point3D(" + self.x + ", " + self.y +
      ", " + self.z + ")";
  }
}

let p3 = new Point3D(3, 4, 12);
say p3.describe(); # Point3D(3, 4, 12)
say p3.distance(); # sqrt(32+42+122) = 13
```

13.5 Super Calls

`super(args...)` calls the parent class constructor. `super.method(args...)` calls a parent method:

Super Calls

```
class Animal {
  fn new(name, age) {
    self.name = name;
    self.age = age;
  }

  fn speak() {
    say self.name + " makes a sound";
  }

  fn birthday() {
    self.age = self.age + 1;
    say self.name + " turned " + self.age + "!";
  }
}

class Dog extends Animal {
  fn new(name, age, breed) {
    super(name, age);      # Animal.new(name, age)
    self.breed = breed;
  }

  fn speak() {
    say self.name + " barks!"; # override
  }
}

let rex = new Dog("Rex", 3, "German Shepherd");
rex.speak();      # Rex barks!
rex.birthday();   # Rex turned 4! (inherited from Animal)
```

13.6 Static Methods

Static methods are defined with `static fn` and are called on the class (not instances). They don't have access to `self`:

Static Methods

```
class MathUtil {
  static fn square(x) { return x * x; }
  static fn cube(x)   { return x * x * x; }
}

say MathUtil.square(7); # 49
say MathUtil.cube(3);   # 27

# Static methods can't be called on instances
let m = new MathUtil();
# m.square(5); # Error static methods are class-only
```

Design Note: OOP Implementation

CRISP's OOP system is implemented as a thin layer over hashes:

- **Classes:** Stored as `Value::Class { name, parent, methods, static_methods }` in the environment. A class is a value like any other it can be assigned to variables, passed to functions, and inspected.
- **Objects:** Stored as `Value::Object { class, fields, methods }`. The object carries a copy of its class's methods (snapshotted at construction time) and a reference to the class name for inheritance chain lookups.
- **Inheritance:** Method resolution walks the parent chain at call time. `super` calls skip the current class and start at the parent, preventing infinite recursion.
- **Self:** Bound automatically by `call_method` when dispatching. `self` is a special variable injected into the method's environment.

This design keeps the interpreter simple while providing the full OOP experience constructors, inheritance, method dispatch, and static methods all built on CRISP's existing hash and function primitives.

14 Modules

CRISP supports modular code organization with the `use` keyword. Currently, `use posix;` is the only built-in module.

Modules

```
# Load the POSIX module
use posix;

# POSIX functions live under a namespaced hash
let pid = posix["fork"]();
let fd = posix["open"]("/tmp/test.txt", O_WRONLY | O_CREAT, 0
    o644);
posix["write"](fd, "Hello!\n", 7);
posix["close"](fd);

# posix is a regular Hash you can inspect it
for func in posix {
    say func; # lists all POSIX function names
}
```

Design Note: Module System

The module system is under active development. Currently, only `use posix;` is supported. Future releases will extend this to user-defined modules (`use mymodule;`), import paths, and selective imports. The namespaced hash pattern (each module creating its own hash) will be the standard approach for all modules.

15 System Programming with POSIX

CRISP provides full POSIX system call integration via an opt-in module. Unlike other scripting languages where system calls pollute the global namespace, CRISP loads POSIX on demand and namespaces all functions under a `posix` hash.

15.1 Loading the POSIX Module

Loading POSIX

```
# Load the POSIX module (opt-in, namespaced)
use posix;

# POSIX functions live under the posix hash
say posix["getpid"](); # prints current PID

# Console read() is NOT shadowed!
let input = readline("> ");
say input;
```

15.2 Process Management

Process Management

```
use posix;

# Fork a child process
let pid = posix["fork"]();
if pid == 0 {
  # Child process
  say "Child PID: ", posix["getpid"]();
  posix["execvp"]("ls", "-la");
} else {
  # Parent process
  say "Parent waiting for child: ", pid;
  let status = posix["waitpid"](pid, 0);
  say "Child exited with: ", status;
}

# Get process IDs
say "PID: ", posix["getpid"]();
say "PPID: ", posix["getppid"]();
```

15.3 File Operations (POSIX)

POSIX File Operations

```
use posix;

# Low-level file descriptor operations
let fd = posix["open"]("/tmp/test.txt",
    posix["O_WRONLY"] | posix["O_CREAT"], 0o644);
posix["write"](fd, "Hello POSIX!\n", 13);
posix["close"](fd);

# Write to file
let fd = posix["open"]("data.txt",
    posix["O_WRONLY"] | posix["O_CREAT"], 0o644);
posix["write"](fd, "Hello POSIX!\n", 13);
posix["close"](fd);

# Seek in file
let fd2 = posix["open"]("data.bin", posix["O_RDONLY"]);
posix["lseek"](fd2, 0, posix["SEEK_END"]);
let size = posix["lseek"](fd2, 0, posix["SEEK_CUR"]);
```

15.4 Signals

Signals

```
use posix;

# Send signal to a process
posix["kill"](pid, posix["SIGTERM"]);

# Set an alarm
posix["alarm"](5); # SIGALRM in 5 seconds
```

15.5 Users and Environment

Users and Environment

```
use posix;

# User/group IDs
say "UID: ", posix["getuid"]();
say "EUID: ", posix["geteuid"]();
say "GID: ", posix["getgid"]();

# Environment variables
let path = posix["getenv"]("PATH");
say "PATH: ", path;

posix["setenv"]("CRISP_TEST", "hello");
say posix["getenv"]("CRISP_TEST"); # hello

# Get all environment variables as hash
let env = posix["environ"]();
say env;
```

15.6 POSIX Function Reference

Table 9: POSIX Functions

Category	Function	Signature	Description
Processes			
	fork	posix["fork"]() Int	Create child process
	execvp	posix["execvp"](file, args...)	Execute program
	waitpid	posix["waitpid"](pid, opts) Int	Wait for child
	spawn	posix["spawn"](cmd, args...)	Spawn process
	getpid	posix["getpid"]() Int	Current PID
	getppid	posix["getppid"]() Int	Parent PID
Signals			
	kill	posix["kill"](pid, sig)	Send signal
	alarm	posix["alarm"](secs)	Schedule alarm
Users			
	getuid	posix["getuid"]() Int	Real UID
	geteuid	posix["geteuid"]() Int	Effective UID
	getgid	posix["getgid"]() Int	Real GID
	getegid	posix["getegid"]() Int	Effective GID
Files			
	open	posix["open"](path, flags, mode?) Int	Open file
	close	posix["close"](fd)	Close file
	read	posix["read"](fd, count) Str	Read bytes
	write	posix["write"](fd, data, len) Int	Write bytes
	lseek	posix["lseek"](fd, offset, whence) Int	Seek position
Env			
	getenv	posix["getenv"](name) Str	Get env var
	setenv	posix["setenv"](name, value)	Set env var
	environ	posix["environ"]() Hash	All env vars

Design Note: Namespaced POSIX

CRISP's decision to namespace POSIX under the `posix` hash solves the classic scripting language problem of namespace pollution. Languages like Perl and PHP expose hundreds of global functions, creating naming conflicts with user code. In CRISP:

- **Opt-in:** POSIX is only loaded when `use posix;` is explicitly invoked
- **Namespaced:** All functions live under `posix["fn"]`, never in the global scope
- **No shadowing:** Console `read()` and POSIX `posix["read"]()` coexist without conflict
- **Discoverable:** `posix` is a plain Hash iterate it with `for` or inspect with `keys()`

This pattern will extend to future modules: `use network;` creates a `network` hash, `use json;` creates a `json` hash, etc.

16 Console Input and Output

CRISP provides type-safe console I/O with proper flush semantics. Output functions support both bare syntax (Perl-style) and parenthesized syntax (Python-style).

16.1 Output Functions

Output Functions

```
# print output without newline (bare syntax)
print "Hello, ";
print "World!";
# Output: Hello, World!

# print output without newline (paren syntax)
print("Hello, ", "World!");
# Output: Hello, World!

# say output with automatic newline (bare syntax)
say "Hello, CRISP!";
# Output: Hello, CRISP!\n

# say output with automatic newline (paren syntax)
say("Hello, ", "CRISP!");
# Output: Hello, CRISP!\n

# Multiple arguments
say "Name: ", name, ", Age: ", age;

# warn output to stderr
warn("Something might be wrong");
# Output (to stderr): Warning: Something might be wrong
```

16.2 Input Functions

Three input functions are available, each with different behavior:

Table 10: Input Functions

Function	Returns	Flushes?	Description
<code>readline(prompt?)</code>	Always String	Yes	Reads a line, strips trailing newline
<code>read(prompt?)</code>	Int, Float, or String	Yes	Auto-detects numeric types
<code>input(prompt?)</code>	Always String	Yes	Python-compatible alias for <code>readline</code>

Input Functions

```
# readline  always returns String
let name = readline("What's your name? ");
say "Hello, ", name, "!";

# read  auto-detects Int, Float, or String
let age = read("How old are you? ");
match age {
  n where type(n) == "int" => { say "Age: ", n; }
  n where type(n) == "float" => { say "Weird age: ", n; }
  _ => { say "That's not a number!"; }
}

# input  Python-compatible alias
let color = input("Favorite color? ");
say "You like ", color;

# Empty input (just press Enter) returns empty string
let data = readline("> ");
if data == "" {
  say "No input provided.";
}

# EOF (Ctrl+D) returns empty string
let eof = readline();
if eof == "" {
  say "Goodbye!";
}
```

16.3 Flush Semantics

All input functions flush stdout before blocking on stdin. This guarantees the prompt is visible before the program waits for input:

Flush Guarantee

```
# This works correctly  prompt appears before blocking
let name = readline("Name: ");
# User sees: Name: _ (cursor waits here)

# Without flush, the prompt might be buffered and invisible
# CRISP guarantees flush, so this never happens
```

16.4 Type Conversion Functions

Type Conversion

```
# int()  parse string to integer, returns null on failure
let n = int("42");
say n;      # 42 (Int)

let bad = int("hello");
say bad;    # null

# float()  parse string to float, returns null on failure
let f = float("3.14");
say f;      # 3.14 (Float)

let bad2 = float("world");
say bad2;   # null

# type()  get the type of any value
say type(42);      # "int"
say type("hello"); # "string"
say type([1, 2]);  # "array"
say type(null);    # "null"
say type(true);    # "bool"

# len()  length of collection or string
say len("CRISP");      # 5
say len([1, 2, 3, 4, 5]); # 5
say len({ a => 1, b => 2 }); # 2
```

17 Multi-Dimensional Arrays

CRISP supports nested array literals, chained indexing, and nested iteration all building on the existing array primitives without special syntax or additional types.

17.1 Nested Array Literals

Arrays can contain other arrays, enabling matrix-like and jagged structures:

Nested Arrays

```
# Single-line nested array
let @matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];

# Multi-line nested array
let @grid = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

# Jagged arrays (rows of different lengths)
let @jagged = [[1, 2], [3, 4, 5], [6]];
```

17.2 Chained Indexing

Postfix indexing chains naturally for multi-dimensional access:

Chained Indexing

```
let @matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];

# Single index returns a row (an array)
say @matrix[0];      # [1, 2, 3]

# Chained indexing drills into nested arrays
say @matrix[1][2];   # 6

# Deep nesting
let @cube = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]];
say @cube[0][1][0];  # 3
```


17.3 Nested Iteration

for loops iterate over each level, enabling nested traversal:

Nested For Loops

```
# Print each cell of a matrix
for row in @matrix {
  for cell in row {
    say cell;
  }
}
# Output: 1 2 3 4 5 6 7 8 9

# Collect all elements into a flat array
let @flat = [];
for row in @matrix {
  for cell in row {
    @flat.push(cell);
  }
}
say @flat; # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Design Note: No Special Matrix Type

CRISP does not have a special matrix or tensor type. Multi-dimensional arrays are simply arrays of arrays. This design choice provides:

- **Simplicity:** No new types, syntax, or concepts to learn
- **Flexibility:** Jagged arrays (rows of different lengths) are natural
- **Composability:** All existing array methods (`.map()`, `.filter()`, `.contains()`) work on nested arrays without modification

Future releases may add dedicated linear algebra operations (`transpose`, `reshape`, `flatten`) as array methods.

18 JSON Serialization

JSON (JavaScript Object Notation) is the lingua franca of modern data exchange. Every web API, configuration file, and data pipeline speaks JSON. CRISP provides native JSON support with four functions that cover the full lifecycle: serialization, pretty-printing, deserialization, and validation.

18.1 Serializing to JSON

The `to_json` function converts any CRISP value into a compact JSON string. Hashes become JSON objects, arrays become JSON arrays, and scalar types map to their JSON equivalents:

JSON Serialization

```
# Build a configuration object
let config = {
  name => "CRISP",
  version => "0.1.5",
  features => ["oop", "regex", "json", "networking"],
  settings => { debug => true, port => 8080 }
};

# Compact JSON
let compact = to_json(compact);
say compact;
# {"features":["oop","regex","json","networking"],"name":"CRISP",
#  "settings":{"debug":true,"port":8080},"version":"0.1.5"}

# Pretty-printed JSON (indented, multi-line)
say to_json_pretty(config);
# {
#   "features": ["oop", "regex", "json", "networking"],
#   "name": "CRISP",
#   "settings": {
#     "debug": true,
#     "port": 8080
#   },
#   "version": "0.1.5"
# }
```

18.2 Deserializing from JSON

`from_json` parses a JSON string into native CRISP values. JSON objects become hashes (with dot-notation field access), arrays become CRISP arrays, and scalar values map directly:

JSON Parsing

```
# Parse a JSON string
let data = from_json('{ "x": 10, "y": 20, "label": "point" }');

# Access fields with dot notation
say data.x;          # 10
say data.y;          # 20
say data.label;      # "point"

# Parse a JSON array
let items = from_json('[ 1, 2, 3, 4, 5 ]');
say items.len();     # 5
say items[2];        # 3
```

18.3 Validation

`json_valid` checks whether a string is syntactically valid JSON without parsing it into values. Use it to validate input before processing:

JSON Validation

```
# Valid JSON
say json_valid('{ "a": 1 }');    # true
say json_valid('[ 1, 2, 3 ]');   # true
say json_valid('"hello"');       # true
say json_valid('42');            # true

# Invalid JSON
say json_valid('not json');      # false
say json_valid('{bad}');         # false
say json_valid('[ 1, 2, ]');     # false (trailing comma)
```

18.4 A Complete Example: Configuration Loader

Here is a practical program that reads a JSON configuration file, validates it, and uses the settings to control program behavior:

JSON Configuration Loader (config_loader.csp)

```
# config_loader.csp  Load and validate a JSON config file

fn load_config(path) {
  if !file_exists(path) {
    say "Error: config file '" + path + "' not found.";
    exit(1);
  }

  let raw = read_file(path);

  if !json_valid(raw) {
    say "Error: invalid JSON in '" + path + "'.";
    exit(1);
  }

  let config = from_json(raw);
  return config;
}

# Load and display
let config = load_config("app.json");

say "App: " + config.name;
say "Version: " + config.version;

# Iterate over features
say "Features:";
for feature in config.features {
  say "  - " + feature;
}

# Use nested settings
if config.settings.debug {
  say "Debug mode: ON";
  say "Port: " + config.settings.port;
  say "Full config (pretty):";
  say to_json_pretty(config);
}
```

18.5 JSON as a Data Exchange Format

One of the most common scripting tasks is transforming data between formats. Here is a program that reads a JSON file, filters and transforms the data, and writes the result back as JSONa pattern used daily in data pipelines:

JSON Transform Pipeline (json_transform.csp)

```
# json_transform.csp  Filter and transform JSON data

# Sample data: list of users
let users_json = '[{"name": "Alice", "age": 25, "active": true},
                  {"name": "Bob", "age": 17, "active": false},
                  {"name": "Charlie", "age": 30, "active": true},
                  {"name": "Diana", "age": 22, "active": true}]';

let users = from_json(users_json);

# Filter: only active users 18+
let adults = users.filter(|u| => u.active && u.age >= 18);

# Transform: add a display_name field
let enriched = adults.map(|u| => {
  u.display_name = u.name + " (" + u.age + ")";
  return u;
});

# Output: pretty-printed JSON
say to_json_pretty(enriched);
```

Why JSON Matters in CRISP

JSON support in CRISP follows the language's design philosophy: simple, composable, and practical. The four functions `to_json`, `to_json_pretty`, `from_json`, and `json_valid` cover the complete JSON lifecycle without ceremony. Combined with CRISP's native hash and array operations, JSON data is immediately usable: parse it, filter it with `.filter()`, transform it with `.map()`, and write it back. No intermediate classes, no boilerplate, no impedance mismatch.

This is the same pattern that made Perl the king of text processing: read, transform, write. CRISP extends this pattern to structured data, making it equally natural to work with JSON APIs, configuration files, and data pipelines.

19 Time and Date Functions

Working with time is essential in scripting: timestamps for logging, date formatting for reports, scheduling, and measuring elapsed time. CRISP provides six time functions that cover Unix timestamps, human-readable dates, formatting, and parsing all without external dependencies.

19.1 Getting the Current Time

Three functions return the current time at different resolutions:

Table 11: Time Functions

Function	Returns	Resolution	Example Output
<code>time()</code>	Int	Seconds	1784987128
<code>timestamp()</code>	Int	Milliseconds	1784987128638
<code>datetime()</code>	String	Seconds	"2026-07-25 15:45:28"
<code>datetime_utc()</code>	String	Seconds	"2026-07-25 13:45:28 UTC"

Basic Time Functions

```
# Unix timestamp  seconds since epoch (Jan 1, 1970)
let now = time();
say "Seconds since epoch: " + now;

# Millisecond timestamp  for high-resolution timing
let ms = timestamp();
say "Milliseconds since epoch: " + ms;

# Human-readable local time
say "Local: " + datetime();
# "Local: 2026-07-25 15:45:28"

# Human-readable UTC time
say "UTC:    " + datetime_utc();
# "UTC:     2026-07-25 13:45:28 UTC"
```

19.2 Formatting Dates

`strftime` formats a Unix timestamp into a human-readable string using format specifiers. This is the same `strftime` familiar to C, Perl, and Python programmers:

Date Formatting with `strftime`

```
let now = time();

# Common formats
say strftime("%Y-%m-%d", now);           # 2026-07-25   (ISO date)
say strftime("%H:%M:%S", now);           # 15:45:28   (time)
say strftime("%Y-%m-%d %H:%M:%S", now);  # 2026-07-25 15:45:28 (datetime)
say strftime("%A, %B %d, %Y", now);      # Saturday, July 25, 2026
say strftime("%I:%M %p", now);           # 03:45 PM   (12-hour)

# Custom log format
say strftime("[%Y-%m-%d %H:%M:%S]", now) + " Server started";
# [2026-07-25 15:45:28] Server started
```

19.3 Parsing Dates

`strptime` does the reverse: it parses a date string into a Unix timestamp using the same format specifiers:

Date Parsing with `strptime`

```
# Parse a date string into a timestamp
let ts = strptime("2026-01-15", "%Y-%m-%d");
say ts;  # 1768435200 (Unix timestamp for Jan 15, 2026)

# Verify by formatting back
say strftime("%A, %B %d, %Y", ts);
# Thursday, January 15, 2026

# Parse a full datetime
let ts2 = strptime("2026-07-25 15:45:28", "%Y-%m-%d %H:%M:%S");
say ts2;  # should match time()
```

19.4 A Complete Example: Elapsed Time Calculator

Measuring how long something takes is a classic scripting task. Here is a program that times a computation and reports the result in human-readable form:

Elapsed Time Calculator (timer.csp)

```
# timer.csp  Measure and display elapsed time

fn elapsed(start_ms, end_ms) {
  let diff_ms = end_ms - start_ms;

  if diff_ms < 1000 {
    return diff_ms + " ms";
  }

  let seconds = diff_ms / 1000.0;

  if seconds < 60 {
    return seconds + " seconds";
  }

  let minutes = int(seconds / 60.0);
  let secs_left = seconds - (minutes * 60.0);
  return minutes + " min " + secs_left + " sec";
}

# Time a computation
say "Computing factorial(20)...";
let start = timestamp();

let result = 1;
let i = 1;
while i <= 20 {
  result = result * i;
  i = i + 1;
}

let end = timestamp();

say "factorial(20) = " + result;
say "Took: " + elapsed(start, end);
```


19.5 Date Arithmetic

Here is a practical program that computes age from a birth date and determines the day of the week someone was born:

Date Arithmetic (age_calculator.csp)

```
# age_calculator.csp  Compute age and birth day-of-week

fn calculate_age(birth_date_str) {
  let birth_ts = strptime(birth_date_str, "%Y-%m-%d");
  let now_ts = time();

  # Seconds in a year (approximate)
  let seconds_per_year = 365.25 * 24 * 60 * 60;
  let age = int((now_ts - birth_ts) / seconds_per_year);

  let day_of_week = strftime("%A", birth_ts);

  return {
    age => age,
    day_of_week => day_of_week,
    birth_date => strftime("%B %d, %Y", birth_ts)
  };
}

let info = calculate_age("1990-05-15");
say "Born: " + info.birth_date;
say "Day of week: " + info.day_of_week;
say "Age: " + info.age + " years";
```

Design Note: Why strftime/strptime?

CRISP delegates date formatting and parsing to the C standard library's `strftime` and `strptime` functions. This is a deliberate choice:

- **Familiarity:** The format specifiers (%Y, %m, %d, %H, %M, %S) are identical to C, Perl, Python, Ruby, and PHP. Anyone who has formatted dates in any of these languages already knows CRISP's date formatting.
- **Locale-aware:** %A produces the correct weekday name for the system's locale. On a Czech system, `strftime("%A", ts)` returns "pondlí", not "Monday".
- **No dependencies:** Using the C standard library means zero additional crates, zero build time, and zero security vulnerabilities from third-party date libraries.

The trade-off is that `strptime` is not available on Windows (it's a POSIX extension). For cross-platform date parsing, future CRISP releases will provide a pure-Rust alternative.

20 Network Programming

CRISP provides network I/O through high-level convenience functions for the most common tasks: fetching web resources, sending data to servers, and working with URLs.

20.1 Fetching Web Resources

The `http_get` function fetches a URL and returns the response body as a string:

HTTP GET

```
# Fetch a web page
let html = http_get("https://example.com");
say "Fetched " + html.len() + " bytes";

# Fetch JSON from an API
let response = http_get("https://api.github.com/repos/user/crisp");
if json_valid(response) {
  let data = from_json(response);
  say "Stars: " + data.stargazers_count;
  say "Forks: " + data.forks_count;
}
```

20.2 Sending Data

`http_post` sends data to a URL with a specified content type:

HTTP POST

```
# Send JSON data
let payload = to_json({
  name => "CRISP",
  version => "0.1.5"
});

let response = http_post("https://httpbin.org/post",
  payload,
  "application/json");

say "Server responded: " + response;
```

20.3 URL Encoding and Decoding

URL Encoding

```
# Encode special characters for URLs
say url_encode("hello world");      # hello%20world
say url_encode("name=CRISP&v=1");    # name%3DCRISP%26v%3D1

# Decode URL-encoded strings
say url_decode("hello%20world");     # hello world
say url_decode("name%3DCRISP");      # name=CRISP
```

20.4 A Complete Example: Simple Web Client

Web Client (web_client.csp)

```
# web_client.csp  Fetch and display web resources

fn fetch_json_api(url) {
  say "Fetching: " + url;
  let raw = http_get(url);

  if raw == "" {
    say "Error: no response from server.";
    return null;
  }

  if !json_valid(raw) {
    say "Error: response is not valid JSON.";
    return null;
  }

  return from_json(raw);
}

# Try fetching a public API
let data = fetch_json_api("https://api.github.com/repos/rust-
lang/rust");

if data != null {
  say "Repository: " + data.full_name;
  say "Description: " + data.description;
  say "Stars: " + data.stargazers_count;
  say "Language: " + data.language;
}
```

20.5 Network Function Reference

Table 12: Network Functions

Function	Signature	Description
<code>http_get</code>	<code>http_get(url) String</code>	Fetch a URL via GET, return response
<code>http_post</code>	<code>http_post(url, body, content_type?) String</code>	POST data to a URL
<code>url_encode</code>	<code>url_encode(s) String</code>	Percent-encode special characters
<code>url_decode</code>	<code>url_decode(s) String</code>	Decode percent-encoded string

Design Note: HTTP Support

CRISP's network module provides simple, blocking HTTP operations via the `ureq` crate a minimal, dependency-light HTTP client. The design philosophy is:

- **Simple by default:** `http_get(url)` does one thing: fetches a URL and returns the body. No callbacks, no streams, no complexity.
- **Opt-in complexity:** Future releases will add streaming responses, async I/O, and WebSocket support for advanced use cases.
- **Self-contained:** The network functions are available globally (no `use network;` required) because web access is a fundamental scripting need, not a specialized system concern.

For low-level socket programming, CRISP delegates to the POSIX module (`use posix; posix["socket"], posix["connect"], etc.`), giving you full control when needed.

21 Performance

Benchmark Results:

- **Startup time:** < 50 ms (warm)
- **REPL responsiveness:** < 10 ms
- **Arithmetic operations:** $\sim 10^6$ ops/sec
- **Function calls:** $\sim 10^5$ calls/sec
- **Memory usage:** ~ 2 MB baseline

22 Comparison with Other Languages

Table 13: Language Comparison

Feature	CRISP	Perl	Python	Rust	Bash
Regex	✓	✓	✗	✗	✓
Sigils	Optional	Required	✗	✗	✗
Memory Safety	✓	✗	✗	✓	✗
One-liners	✓	✓	✗	✗	✓
POSIX	✓	✓	✗	✓	✓
Lambda	✓	✓	✓	✓	✗
OOP	✓	✓	✓	✗	✗
REPL	✓	✗	✓	✗	✗
Fast Startup	✓	✓	✗	✗	✓
JSON	✓	✓	✓	✗	✗
HTTP Client	✓	✓	✓	✗	✗

23 Future Directions

CRISP continues to evolve with planned features:

Planned Features

- **Bytecode VM:** Improved performance through bytecode compilation
- **JIT Compilation:** Just-in-time compilation for hot code paths
- **Async/Await:** Native asynchronous programming support
- **Multi-threading:** Concurrent execution with thread safety
- **GUI Framework:** Native GUI applications with GTK/Qt
- **TUI Library:** Terminal user interfaces with Ratatui
- **WebAssembly:** Compilation to WASM for browser deployment
- **Package Manager:** CRISPAN for dependency management
- **VS Code Extension:** Language Server Protocol (LSP) support
- **Windows Support:** Full cross-platform compatibility

24 Conclusion

CRISP successfully combines Perl's expressiveness with Rust's safety. It provides a modern scripting experience with:

- **Memory safety** guaranteed by Rust
- **Perl-inspired** text processing and functional tools
- **System access** through POSIX system calls
- **Interactive development** via REPL
- **One-liners** for quick scripting
- **Object-oriented programming** with classes, inheritance, and static methods
- **Modern error handling** with try/catch/finally

Final Assessment

CRISP represents a significant step forward in scripting language design, bridging the gap between expressiveness and safety. It is suitable for:

- System administration scripts
- Text processing pipelines
- Quick prototyping
- Educational purposes
- Production system scripting

"Keep your code CRISP and clean."

25 Standard Library Reference

25.1 Console I/O

- `print(args...)` Output without newline. Supports bare and paren syntax.
- `say(args...)` Output with newline. Supports bare and paren syntax.
- `warn(args...)` Output to `STDERR` with "Warning:" prefix.
- `readline(prompt?)` `String` Read a line from `stdin`, strips trailing newline.
- `read(prompt?)` `Int` | `Float` | `Str` Like `readline` but auto-detects type.
- `input(prompt?)` `String` Python-compatible alias for `readline`.

25.2 Error Handling

- `die(msg)` Emit catchable `UserError`.
- `throw(msg)` Alias for `die` (catchable).
- `assert(cond, msg?)` Testing assertion.

25.3 Process Control

- `exit(code?)` Terminate process with exit code (default 0). `finally` blocks execute before termination. Not catchable.
- `_exit(code?)` Hard exit via `std::process::exit`. No cleanup, immediate termination.

25.4 Type Conversion and Introspection

- `int(value)` `Int` | `Null` Parse string to integer, returns null on failure.
- `float(value)` `Float` | `Null` Parse string to float, returns null on failure.
- `type(value)` `String` Returns type name ("int", "float", "string", "array", "hash", "bool", "null", "ref", "function", "object", "class").
- `len(value)` `Int` Length of string, array, or hash.

25.5 Array Operations

- `push(arr, val)` Append to array.
- `pop(arr)` `Value` Remove and return last element.
- `shift(arr)` `Value` Remove and return first element.
- `unshift(arr, val)` Prepend to array.
- `map(arr, callback)` `Array` Transform each element.
- `filter(arr, callback)` `Array` Keep matching elements.
- `grep(arr, callback)` `Array` Alias for `filter`.
- `sort(arr)` `Array` Sort array.
- `take(arr, n)` `Array` First N elements.
- `zip(arr1, arr2)` `Array` Pair elements from two arrays.

25.6 Math and Random

- `rand(max?)` Int Random integer in `0..max-1` (default: `u64::MAX`).
- `sqrt(x)` Float Square root.
- `pow(base, exp)` Float Exponentiation.
- `abs(x)` Float Absolute value.
- `min(a, b, ...)` Float Minimum value.
- `max(a, b, ...)` Float Maximum value.
- `PI`, `E`, `TAU` Mathematical constants.

25.7 File I/O (High-Level)

- `read_file(path)` String Read entire file.
- `write_file(path, content)` Bool Write string to file.
- `file_exists(path)` Bool Check if file exists.

25.8 JSON

- `to_json(value)` String Serialize to JSON string.
- `to_json_pretty(value)` String Pretty-printed JSON.
- `from_json(string)` Value Parse JSON string.
- `json_valid(string)` Bool Check if string is valid JSON.

25.9 Crypto

- `md5(s)` String MD5 hash (hex).
- `sha1(s)` String SHA-1 hash (hex).
- `sha256(s)` String SHA-256 hash (hex).
- `sha512(s)` String SHA-512 hash (hex).
- `base64_encode(s)` String Base64 encode.
- `base64_decode(s)` String Base64 decode.
- `random_bytes(n)` String Random hex bytes.

25.10 Time

- `time()` Int Unix timestamp (seconds).
- `timestamp()` Int Unix timestamp (milliseconds).
- `datetime()` String Local datetime string.
- `datetime_utc()` String UTC datetime string.
- `strftime(fmt, ts)` String Format timestamp.
- `strptime(s, fmt)` Int Parse date string to timestamp.

25.11 Network

- `http_get(url)` String Fetch URL via HTTP GET.
- `http_post(url, body, content_type?)` String POST data to URL.
- `url_encode(s)` String Percent-encode string for URLs.
- `url_decode(s)` String Decode percent-encoded string.

25.12 Regex (stdlib)

- `regex_match(text, pattern)` Bool Test if pattern matches.
- `regex_replace(text, pattern, replacement)` String Replace matches.
- `regex_split(text, pattern)` Array Split by regex.
- `regex_find_all(text, pattern)` Array All matches.
- `regex_capture(text, pattern)` Array Capture groups.

25.13 Testing

- `assert_eq(a, b, msg?)` Assert equality.
- `assert_ne(a, b, msg?)` Assert inequality.
- `assert_true(cond, msg?)` Assert truth.
- `assert_false(cond, msg?)` Assert falsity.
- `assert_throws(fn)` Assert function throws.

25.14 Method Dispatch (Value Methods)

Array methods: `.contains(val)`, `.push(val)`, `.pop()`, `.shift()`, `.unshift(val)`, `.len()`, `.map(callback)`, `.filter(callback)`, `.grep(callback)`, `.join(sep)`, `.take(n)`, `.zip(other)`
String methods: `.contains(substr)`, `.len()`, `.split(delim)`, `.join(sep)`, `.to_lower()`, `.to_upper()`
Hash methods: `.keys()`, `.values()`, `.len()`, `.contains(key)`

25.15 POSIX Module (Opt-in via `use posix;`)

Processes: `posix["fork"]`, `posix["execvp"]`, `posix["waitpid"]`, `posix["spawn"]`, `posix["getpid"]`, `posix["getppid"]`
Signals: `posix["kill"]`, `posix["alarm"]`
Users: `posix["getuid"]`, `posix["geteuid"]`, `posix["getgid"]`, `posix["getegid"]`
Files: `posix["open"]`, `posix["close"]`, `posix["read"]`, `posix["write"]`, `posix["lseek"]`
Environment: `posix["getenv"]`, `posix["setenv"]`, `posix["environ"]`

25.16 Process Management (stdlib)

- `system(cmd, args...)` Int Run command, return exit code.
- `qx(cmd)` String Capture command output.
- `pid()` Int Current process ID.

26 Keyword Reference

All CRISP keywords with their syntax and behavior.

Table 14: Complete CRISP Keywords v0.1.5

Keyword	Category	Description
<code>let</code>	Declaration	Declares a new variable with optional initialization.
<code>const</code>	Declaration	Declares an immutable constant (must be initialized).
<code>fn</code>	Function	Defines a named function with parameters and body.
<code>return</code>	Control Flow	Exits a function, optionally returning a value.
<code>if</code>	Conditional	Executes a block if condition is true.
<code>else</code>	Conditional	Alternative branch for <code>if</code> . Chains with <code>if</code> for <code>else if</code> .
<code>while</code>	Loop	Repeats a block while condition is true.
<code>for</code>	Loop	Iterates over arrays, hashes, strings, or ranges.
<code>in</code>	Loop	Separates loop variable from iterable in <code>for</code> loops.
<code>break</code>	Loop Control	Exits the innermost loop immediately. Error outside loops.
<code>continue</code>	Loop Control	Skips to the next iteration of the innermost loop. Error outside loops.
<code>match</code>	Pattern Matching	Branches on value against patterns with optional <code>where</code> guards.
<code>where</code>	Guard	Adds a condition to a <code>match</code> arm. The arm fires only if guard is true.
<code>try</code>	Error Handling	Introduces a block whose errors can be caught.
<code>catch</code>	Error Handling	Catches errors from a preceding <code>try</code> block.
<code>finally</code>	Error Handling	Block that always executes after <code>try</code> (and <code>catch</code>), regardless of errors.

Continued on next page

Table 14: Complete CRISP Keywords v0.1.5 (continued)

Keyword	Category	Description
<code>throw</code>	Error Handling	Emits a catchable <code>UserError</code> with a message. Interchangeable with <code>die</code> . <code>throw "invalid input";</code>
<code>die</code>	Error Handling	Emits a catchable <code>UserError</code> with a message. Interchangeable with <code>throw</code> . <code>die "fatal error";</code>
<code>class</code>	OOP	Defines a class with methods and optional parent. <code>class Point { fn new(x, y) { ... } }</code>
<code>extends</code>	OOP	Specifies parent class for inheritance. <code>class Dog extends Animal { ... }</code>
<code>self</code>	OOP	Refers to the current object instance inside methods. <code>self.x = 10;</code>
<code>super</code>	OOP	Calls parent constructor (<code>super(args)</code>) or method (<code>super.method(args)</code>). <code>super(x, y); super.distance();</code>
<code>static</code>	OOP	Declares a static method (called on class, not instance). <code>static fn square(x) { return x * x; }</code>
<code>new</code>	OOP (Contextual)	Contextual keyword for object construction: <code>new ClassName(args).</code> <code>let p = new Point(3, 4);</code>
<code>print</code>	Output	Outputs arguments to stdout without trailing newline. <code>print "Hello, ", name; print("Hello, ", name); (paren)</code>
<code>say</code>	Output	Outputs arguments to stdout with automatic trailing newline. <code>say "Hello, ", name, "!"; say("Hello, ", name); (paren)</code>
<code>warn</code>	Output	Outputs arguments to stderr with "Warning:" prefix. <code>warn("low memory");</code>
<code>use</code>	Module	Loads an opt-in module. Currently <code>posix</code> is the only built-in module. <code>use posix; creates posix hash variable</code>
<code>true</code>	Literal	Boolean true value. <code>let flag = true;</code>
<code>false</code>	Literal	Boolean false value. <code>if done == false { ... }</code>
<code>null</code>	Literal	Represents absence of value. Returned by failed conversions. <code>let x = null; if x == null { ... }</code>

26.1 Control Flow Keywords

Table 15: Control Flow Keywords Internal Behavior

Keyword	Sentinel Error	Behavior
<code>return</code>	<code>RuntimeError::ReturnSignal(Value)</code>	Unwinds call stack. <code>finally</code> blocks execute before propagation.
<code>break</code>	<code>RuntimeError::BreakSignal</code>	Exits innermost loop. Error if used outside <code>while/for</code> .
<code>continue</code>	<code>RuntimeError::ContinueSignal</code>	Skips to next loop iteration. Error if used outside <code>while/for</code> .
<code>throw/die</code>	<code>RuntimeError::UserError(String)</code>	Catchable by <code>catch</code> . Uncaught: terminates script with message.

References

References

- [1] Kernighan, B. W. & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- [2] Prata, S. (2013). *C Primer Plus* (6th ed.). Addison-Wesley.
- [3] Wall, L. et al. (1987). *Programming Perl*. O'Reilly Media.
- [4] Matsakis, N. & Klock, F. (2014). *The Rust Programming Language*. Mozilla.
- [5] Van Rossum, G. (1991). *Python Programming Language*. Python Software Foundation.
- [6] *Logos: Fast Lexical Analysis in Rust*. <https://github.com/maciejhirsz/logos>
- [7] *libc: Rust bindings to libc*. <https://github.com/rust-lang/libc>